

CAHIER DES CHARGES — VERSION MISE À JOUR

PFETracker

Plateforme de suivi des Projets de Fin d'Études

Réalisé par : Fatma Hassine

Établissement : [Nom de l'établissement]

Filière / Niveau : [Filière / Niveau]

Encadrant : [Nom de l'encadrant]

Version : 2.0 — mise à jour du cahier des charges initial, alignée sur le code

Date : 16 septembre 2026

Note méthodologique — pourquoi une version 2.0 : le cahier des charges initial de ce projet n'a pas été retrouvé à jour ; seule une version antérieure était disponible. Le code source contient cependant des commentaires citant explicitement certaines de ses sections (§4.3.3, §4.4.3, §6.3.1, §5.7, §7.1, §7.2...), ce qui confirme qu'il s'agit bien du document de référence initial du projet. La numérotation des sections a donc été conservée à l'identique pour préserver la traçabilité avec le code et avec le rapport sur les outils de gestion des tests. En revanche, le contenu a été vérifié section par section directement dans le code source et corrigé partout où une divergence a été constatée. La section 10 de ce document liste explicitement tous les écarts identifiés entre la version initiale et cette mise à jour.

Table des matières

1. Introduction et contexte général

2. Acteurs du système

3. Création de comptes et mécanisme d'affectation

Module 1 — Gestion des utilisateurs, sécurité et administration (§4)

Module 2 — Gestion des PFE, jalons, tâches et suivi (§5)

Module 3 — Communication, réunions et tableaux de bord (§6)

7. Exigences non fonctionnelles

8. Stack technique et outils

9. Répartition des modules

10. Écarts identifiés par rapport à la version initiale

1. Introduction et contexte général

1.1 Contexte

Le Projet de Fin d'Études (PFE) représente l'aboutissement du parcours académique d'un étudiant. Il mobilise des compétences techniques, méthodologiques et organisationnelles sur une période de plusieurs mois. Son suivi rigoureux est essentiel pour garantir la qualité pédagogique et administrative de la formation.

Dans la réalité actuelle, ce suivi repose sur des outils disparates : emails, messageries instantanées, documents partagés, ou réunions informelles. Il en résulte un manque de traçabilité, une mauvaise visibilité sur l'avancement réel des projets, et une charge administrative élevée pour les encadrants et l'administration.

1.2 Problématique

- Absence d'un espace centralisé pour suivre les tâches et livrables d'un PFE
- Communication informelle et non traçable entre étudiant et encadrant
- Impossibilité pour l'encadrant d'avoir une vue globale sur tous ses étudiants simultanément
- L'administration ne dispose d'aucun outil pour superviser l'avancement des PFE à l'échelle du département
- Le directeur des stages ne peut pas comparer les performances entre départements
- Aucun mécanisme automatique de détection des retards ou d'inactivité prolongée

1.3 Solution proposée — PFETracker

PFETracker est une application web collaborative développée avec **React 18 (TypeScript, Vite)** pour le frontend et **Spring Boot 3.2 (Java 17)** pour le backend, structurée en trois modules indépendants. Elle couvre l'intégralité du cycle de vie d'un PFE : de la création du compte jusqu'à la clôture du projet, en passant par le suivi des tâches, la communication, la planification des réunions et la supervision administrative.

2. Acteurs du système

PFETracker implique six catégories d'acteurs humains, plus un acteur automatique, avec des niveaux d'accès et des responsabilités bien distincts. Les rôles listés ci-dessous correspondent exactement à l'énumération réelle du code (entity/module1/enums/Role.java).

Acteur	Rôle Spring	Responsabilité	Périmètre d'accès
Étudiant	ROLE_ETUDIANT	Suit son PFE, soumet ses livrables, communique avec son encadrant	Son PFE uniquement
Encadrant	ROLE_ENCADRANT	Supervise les PFE de ses étudiants, valide les tâches, planifie les réunions	PFE de ses étudiants affectés
Responsable de département	ROLE_CHEF_DEPARTEMENT	Gère les comptes de son département, supervise l'avancement global	Son département uniquement
Directeur des stages	ROLE_DIRECTEUR	Supervise tous les départements, accès aux indicateurs consolidés	Tous les départements — lecture seule
Service des stages *	ROLE_SERVICE_STAGE	Suit les stages et conventions, alertes d'expiration, tableau de bord dédié	Tous les stages en cours
Administrateur *	ROLE_ADMIN	Administration système globale, gestion des comptes et départements, réinitialisation système	Tout le système
Système (automatique)	—	Envoie les notifications, détecte les retards, génère les alertes	Interne — aucune interface utilisateur

** Acteurs absents de la version initiale du cahier des charges, mais bien présents et fonctionnels dans le code (AdminController, ServiceDesStagesController) — voir §10.*

2.1 Hiérarchie administrative

L'établissement comporte plusieurs départements. La hiérarchie administrative est organisée comme suit :

- L'administrateur dispose d'un accès global au système : création/suppression de comptes et de départements, réinitialisation système, consultation de tous les journaux d'audit.
- Le directeur des stages supervise l'ensemble des départements et a une vision globale consolidée de tous les PFE, en lecture seule.
- Le service des stages suit transversalement les conventions et périodes de stage, indépendamment du découpage par département.
- Chaque département dispose d'un responsable de département qui gère les comptes, les affectations et les indicateurs de son périmètre.
- Les encadrants et étudiants appartiennent chacun à un département spécifique. Un encadrant ne peut encadrer que des étudiants de son propre département.

3. Création de comptes et mécanisme d'affectation

3.1 Création de compte — Flux unifié

La création de compte est identique pour les étudiants et les encadrants. Elle repose sur l'email institutionnel comme mécanisme de contrôle d'accès principal (domaine @enicar.ucar.tn). Aucune création manuelle par l'admin n'est nécessaire pour ces deux rôles.

#	Action	Responsable
1	L'utilisateur saisit son email institutionnel (@enicar.ucar.tn) et choisit son rôle (étudiant ou encadrant)	Utilisateur
2	Le système valide le domaine email — tout email hors domaine institutionnel est rejeté	Système
3	Un mot de passe aléatoire sécurisé (12 caractères) est généré et envoyé par email	Système
4	L'utilisateur se connecte avec le mot de passe reçu	Utilisateur
5	Le flag mustChangePassword=true redirige obligatoirement vers la page de changement de mot de passe	Système
6	L'utilisateur choisit son nouveau mot de passe — le flag passe à false	Utilisateur
7	Le compte — étudiant ET encadrant — reste à l'état enabled=false, en attente de validation. Le responsable de département valide en 1 clic.	Responsable dept.

Correction par rapport à la version initiale : le document d'origine ne soumettait à validation que les comptes encadrant. Le code applique en réalité la même règle aux étudiants et aux encadrants (AuthServiceImpl, cf. §3.1 et §4.3.1).

3.2 Mécanisme d'affectation encadrant-étudiant

Écart majeur avec la version initiale : le mécanisme d'invitation « façon Google Classroom » (code encadrant ENC-XXXXX envoyé par email, acceptation par l'étudiant) décrit dans la version initiale a été retiré du système. Un commentaire explicite dans InvitationController.java l'indique : cette fonctionnalité n'est plus utilisée, pour des raisons de confidentialité. Le champ code existe encore en base sur les entités Encadrant/Étudiant, mais il n'est plus exploité pour créer une affectation.

Le mécanisme réellement actif aujourd'hui repose sur deux voies, toutes deux pilotées par le responsable de département — l'étudiant et l'encadrant n'initient plus eux-mêmes la relation :

Affectation manuelle : le responsable de département sélectionne un étudiant non affecté (« orphelin ») et un encadrant disponible, puis valide l'affectation directement (ResponsableController).

Import en masse : les affectations encadrant-étudiant peuvent être importées en une seule fois depuis un fichier Excel/CSV (PfeImportController), utile en début d'année pour affecter une promotion entière.

Règles métier confirmées et toujours en vigueur :

- Un étudiant ne peut être lié qu'à un seul encadrant à la fois.
- Un encadrant peut superviser plusieurs étudiants, dans la limite d'un quota configurable par le responsable de département (défaut : 5).
- Le responsable de département peut forcer, rompre ou réaffecter une relation en cas de problème exceptionnel (ex. départ d'un encadrant).
- Une vérification automatique des étudiants non affectés se déclenche après 21 jours (DirecteurController.verifierEtAffecter).

Module 1 — Gestion des utilisateurs, sécurité et administration

Ce module constitue la fondation technique de PFETracker. Il gère l'ensemble du cycle de vie des utilisateurs, la sécurisation des accès via JWT et Spring Security, ainsi que toutes les fonctionnalités des acteurs administratifs (responsable de département, directeur des stages, service des stages, administrateur).

Stack : Spring Boot · Spring Security · JWT (HS256) · BCrypt · JavaMailSender · React · React Router

4.1 Authentification et sécurité

4.1.1 Inscription et activation

- Formulaire d'inscription : saisie de l'email institutionnel (@enicar.ucar.tn) et du rôle souhaité
- Validation du domaine email côté backend — rejet immédiat de tout email non institutionnel
- Génération d'un mot de passe aléatoire sécurisé (12 caractères), envoyé par email
- Mot de passe stocké hashé avec BCrypt
- Flags mustChangePassword=true et enabled=false stockés en base à la création
- Redirection forcée vers la page de changement de mot de passe à la première connexion
- Compte activé (enabled=true) uniquement après validation du responsable de département — pour étudiant et encadrant

4.1.2 Connexion et gestion des sessions

- Connexion par email + mot de passe
- Jeton d'accès JWT signé en **HS256** (algorithme symétrique), durée de vie 15 minutes
- Refresh token, durée de vie 7 jours, stocké en base avec rotation à chaque rafraîchissement
- Déconnexion : révocation du refresh token en base
- Verrouillage du compte après 5 tentatives de connexion échouées consécutives
- Déverrouillage automatique après 30 minutes, ou manuellement par le responsable de département
- Limitation de débit (rate limiting) : 10 tentatives de connexion / minute, 5 inscriptions / minute, par adresse IP

4.1.3 Gestion des rôles et autorisations

Spring Security est configuré avec les six rôles décrits en §2. Chaque endpoint API sensible est protégé par une annotation @PreAuthorize vérifiant le rôle requis. L'isolation entre

départements est stricte : un responsable de département ne peut accéder aux données d'un autre département, et un encadrant ne peut accéder qu'aux PFE de ses propres étudiants affectés.

Point de vigilance technique (non présent dans la version initiale) : le code contient deux énumérations de rôles distinctes et incompatibles : Role (module1, les six valeurs officielles ci-dessus) et UserRole (module3, quatre valeurs en anglais : ROLE_STUDENT, ROLE_SUPERVISOR, ROLE_DEPT_MANAGER, ROLE_DIRECTOR), cette dernière utilisée par 4 classes du module 3 (SecurityUtils, CurrentUserService, PfeService, TaskService). Cette divergence architecturale est une dette technique réelle à corriger, indépendante des exigences fonctionnelles elles-mêmes.

4.1.4 Récupération de mot de passe

- L'utilisateur saisit son email institutionnel sur la page « Mot de passe oublié »
- Un lien de réinitialisation à usage unique est envoyé par email en cas d'email valide
- Réponse générique identique que l'email existe ou non, pour éviter toute énumération de comptes

4.2 Gestion des profils utilisateurs

4.2.1 Profil étudiant

- Informations personnelles et académiques (département, niveau)
- Consultation de l'encadrant affecté et du statut de la relation
- Modification des informations personnelles (email et département modifiables uniquement par l'admin)

4.2.2 Profil encadrant

- Informations professionnelles (grade, spécialité, bureau)
- Liste des étudiants actuellement affectés avec lien vers leur PFE
- Statistiques personnelles : nombre de PFE encadrés, charge actuelle

4.3 Fonctionnalités du Responsable de département

4.3.1 Gestion des comptes

- Tableau de bord listant tous les comptes en attente de validation (étudiants et encadrants)
- Validation en 1 clic ou refus avec motif, notification automatique
- Activation/désactivation, déverrouillage, réinitialisation de mot de passe, changement de département

4.3.2 Gestion des affectations

- Visualisation de toutes les relations encadrant-étudiant du département, identification des orphelins

- Affectation manuelle ou import en masse (voir §3.2), rupture et réaffectation
- Consultation de la charge de chaque encadrant, configuration de la limite maximale (défaut : 5)

4.3.3 Supervision et export

- Indicateurs d'avancement du département (actifs, terminés, en retard, progression moyenne)
- Aucun accès aux messages privés ni au détail des tâches — uniquement les indicateurs
- Export des données du département au format Excel (.xlsx, Apache POI) ou PDF (OpenPDF)

4.4 Fonctionnalités du Directeur des stages et du Service des stages

- Directeur des stages : vue consolidée en lecture seule sur tous les départements, comparatifs inter-départements, export d'un rapport global (Excel/PDF)
- Service des stages : suivi des étudiants en stage, alertes de stages proches de l'expiration ou expirés, tableau de bord dédié

4.5 Journalisation et audit

- Toute action sensible est journalisée : connexion, changement de mot de passe, validation de compte, affectation
- Le responsable de département consulte les logs de son département ; l'administrateur consulte l'ensemble

Module 2 — Gestion des PFE, jalons, tâches et suivi de progression

Ce module est le cœur pédagogique de PFETracker. Il structure le PFE en unités de travail mesurables, gère le cycle de vie complet des tâches et des livrables, et intègre un moteur de détection automatique des retards.

Stack : Spring Boot · Spring Data JPA · MySQL · Spring Scheduler · React

5.1 Initialisation du PFE

- Un PFE est créé automatiquement dès l'affectation encadrant-étudiant, initialisé avec 6 jalons prédéfinis
- L'étudiant complète la fiche projet (titre, description, objectifs, technologies) ; l'encadrant la valide ou demande des modifications
- Statuts globaux du PFE : Initialisé → En cours → (En retard) → Terminé → Soutenu, avec possibilité de Suspendu

5.2 Structure en jalons

Le PFE est structuré en 6 jalons obligatoires : définition du sujet, état de l'art, conception, développement, tests et validation, rapport et soutenance. Chaque jalon a un statut (non commencé / en cours / terminé) et un pourcentage de complétion calculé automatiquement à partir de ses tâches.

5.3 Tâches et sous-tâches

Cycle de vie d'une tâche : non commencé → en cours → soumis → validé / à corriger / annulé. L'historique complet des changements de statut est conservé. Chaque tâche a une priorité (faible/normale/haute/critique) et peut contenir des sous-tâches à cocher.

5.4 Livrables et fichiers

- Upload de fichiers (PDF, DOCX, PPTX, images, ZIP) attachés à une tâche ou un jalon, stockage sur **système de fichiers local** (répertoire uploads/module2)
- Taille maximale : 50 Mo par fichier
- Versioning réel : chaque nouvelle soumission crée une nouvelle version numérotée sans écraser la précédente, nom de fichier unique (UUID)

5.5 Organisation en mini-sprints

L'encadrant crée des sprints pour organiser le travail. L'étudiant soumet un bilan de fin de sprint que l'encadrant valide ou refuse. Le sprint suit un cycle : planifié → en cours → bilan soumis → validé → clôturé.

5.6 Calcul automatique de la progression

La progression d'une tâche (0% / 50% / 100%) se propage automatiquement en moyenne au niveau du jalon, puis en moyenne pondérée au niveau du PFE global, à chaque changement de statut.

5.7 Moteur de détection des retards

Implémenté via Spring @Scheduled, sans dépendance externe. Il s'exécute chaque nuit à 2h00 et applique les règles suivantes :

- Tâche en retard : date limite dépassée et statut ≠ Validé → notification étudiant + encadrant
- Jalon en retard : progression insuffisante à mi-parcours → statut PFE passe à « En retard »
- Stagnation étudiant : aucune mise à jour depuis 7 jours (seuil configurable) → relance automatique
- PFE à risque : écart de progression > 20% par rapport au calendrier attendu
- Sprint non clôturé après 3 jours sans bilan → rappel automatique

Un rapport hebdomadaire par encadrant est généré chaque lundi matin (email + tableau de bord).

Module 3 — Communication, réunions synchrones et tableaux de bord

Stack : Spring Boot · WebSocket (STOMP) · Google Calendar API · React

6.1 Messagerie et commentaires

- Messagerie privée étudiant–encadrant, liée à un PFE, avec pièces jointes, statut lu/non lu, recherche par mot-clé et par plage de dates
- Archivage automatique des messages d'un PFE clôturé et des messages de plus de 90 jours
- Commentaires sur les tâches avec support des mentions « @ »

6.2 Système de notifications

Le système repose sur un modèle de préférences par utilisateur (UserNotificationPreference), plus riche que la matrice fixe d'origine :

- 3 canaux configurables : email, in-app, WebSocket
- 5 types d'événements : messages, réunions, tâches, commentaires, alertes
- Réglages fins par type d'email (ex. email sur mention, email sur rappel de réunion)
- Digest quotidien optionnel à heure configurable
- Centre de notifications limité aux 30 dernières, marquage individuel ou global comme lues

6.3 Planification des réunions

- Création d'une réunion avec génération automatique d'un lien Google Meet via l'API Google Calendar (OAuth2)
- Lien de secours généré automatiquement si l'accès Google n'est pas autorisé
- Acceptation/refus d'invitation, modification, annulation, compte-rendu post-réunion

6.4 Tableaux de bord

Un tableau de bord dédié existe pour chaque rôle : étudiant (progression, deadlines, dernier feedback), encadrant (liste des étudiants, file de validation, alertes), responsable de département (indicateurs, comptes en attente, orphelins), directeur des stages (comparatif inter-départements), et service des stages (suivi des stages).

7. Exigences non fonctionnelles

7.1 Sécurité

- Jetons JWT signés en **HS256** (clé symétrique) — corrigé par rapport à la version initiale qui prévoyait RS256
- Access token 15 minutes, refresh token 7 jours avec rotation
- Verrouillage de compte après 5 échecs, déverrouillage automatique après 30 minutes
- Limitation de débit : 10 connexions/min, 5 inscriptions/min par IP
- Mots de passe hashés BCrypt, validation de toutes les entrées côté serveur

7.2 Performance

- Traitement asynchrone de l'envoi d'email pour ne jamais bloquer la réponse API (cible < 2s)
- Taille maximale de téléversement : 50 Mo par fichier

7.3 Ergonomie et maintenabilité

- Interface responsive, langue principale française
- Architecture backend en couches Controller → Service → Repository
- Documentation API via Swagger / OpenAPI 3 (springdoc-openapi)

Note : les points de cette section 7 reflètent les éléments effectivement vérifiés dans le code (JWT, rate limiting, verrouillage, taille de fichier, architecture en couches, Swagger). D'autres exigences non fonctionnelles usuelles (CSP, protection CSRF, compression gzip...) n'ont pas fait l'objet d'une vérification exhaustive dans cette passe et mériteraient un audit dédié si une certitude totale est requise.

8. Stack technique et outils

Couche	Technologie
Frontend	React 18 · TypeScript · Vite · Radix UI / MUI (au lieu d'Angular initialement prévu)
Backend	Spring Boot 3.2 · Spring Security · Spring Data JPA
Base de données	MySQL 8.3 (au lieu de PostgreSQL initialement prévu)
Authentification	JWT HS256 (au lieu de RS256) · BCrypt
Temps réel	Spring WebSocket + STOMP
Stockage fichiers	Système de fichiers local (au lieu de MinIO initialement prévu)
Réunions	Google Calendar API v3 (génération automatique de liens Google Meet)
Moteur de règles	Spring @Scheduled (gratuit, zéro dépendance externe)
Documentation API	Swagger / OpenAPI 3 (springdoc-openapi)
Tests backend	JUnit 5 · Mockito · Spring Boot Test (disponibles dans le pom.xml, à mettre en œuvre)
Tests frontend	Aucun outil configuré actuellement (ni Jasmine/Karma, ni Vitest/Jest) — recommandation : Vitest + React Testing Library, cohérents avec Vite

9. Répartition des modules

Le projet est structuré en trois modules pensés pour être développés de façon indépendante — le Module 1 en premier, car les Modules 2 et 3 dépendent de son système d'authentification.

Module 1 — Authentification JWT, rôles, création de comptes, affectation, tableaux de bord admin/directeur/service des stages, journalisation. Aucune dépendance.

Module 2 — PFE, jalons, tâches, livrables, sprints, calcul de progression, moteur de règles. Dépend des API d'authentification du Module 1.

Module 3 — Messagerie, commentaires, notifications temps réel, réunions Google Meet, tableaux de bord étudiant/encadrant. Dépend du Module 1 (auth) et du Module 2 (données PFE).

10. Écarts identifiés par rapport à la version initiale

Cette section liste explicitement, à titre de traçabilité, tous les écarts constatés entre le document initial et l'implémentation réelle vérifiée dans le code.

Sujet	Version initiale	Implémentation réelle
Frontend	Angular	React 18 + TypeScript + Vite
Base de données	PostgreSQL	MySQL 8.3
Algorithme JWT	RS256 (asymétrique)	HS256 (symétrique)
Stockage fichiers	MinIO (self-hosted)	Système de fichiers local
Affectation encadrant–étudiant	Invitation par code (ENC-XXXXXX), style Google Classroom	Désactivée — affectation manuelle par le responsable de département ou import en masse
Acteurs	4 rôles humains (pas d'admin, pas de service des stages)	6 rôles : + Administrateur, + Service des stages
Validation de compte	Encadrant uniquement	Étudiant ET encadrant
Tests frontend	Jasmine / Karma / Angular Testing Library	Aucun outil configuré (recommandation : Vitest + RTL)
Notifications	Matrice fixe événement→canal	Préférences par utilisateur, plus granulaires, + digest quotidien
Nommage des rôles (interne)	Non applicable	Deux énumérations de rôles incompatibles coexistent (module1 vs module3) — dette technique